YÖNETİCİ ÖZETİ & KAPSAMLI ARAŞTIRMA RAPORU

Doğal Dil İşleme: Tokenizer Eğitimi ve Kırmızı Takım (Red-Teaming) Bulguları

Hugging Face LLM Course Temelinde Türkçe Üzerine Derinlemesine Zafiyet Analizi

1. Yönetici Özeti

Doğal Dil İşleme (NLP) ekosisteminde modellerin metinleri anlayabilmesi, kelimelerin veya kelime parçalarının sayısal dizilere dönüştürülmesiyle mümkündür. Bu dönüşüm işlemine **Tokenizasyon (Jetonlaştırma)**, bu işlemi gerçekleştiren algoritmaya ise **Tokenizer** adı verilir. Modern Transformer modellerinin başarısının ardındaki en büyük gizli kahramanlar tokenizer'lardır.

Hugging Face LLM eğitim serisinin 6. Bölümündeki temel iddia şudur: *Başka bir dil (örneğin İngilizce) için eğitilmiş bir tokenizer, yeni bir dilde (örneğin Türkçe) her zaman kötü ve verimsiz performans gösterir*. Bu rapor, söz konusu iddiayı Türkçe özelinde ölçülebilir hale getirmek ve endüstride sıklıkla karşılaşılan "hazır modellerin Türkçeye uyarlanması" senaryolarında ortaya çıkan sessiz hataları ortaya koymak amacıyla hazırlanmıştır.

Sessiz Zafiyet Tehlikesi: Yapılan Red-Teaming (Kırmızı Takım) testlerinin ortak bulgusu, tespit edilen tüm dil/tokenizer uyuşmazlığı hatalarının *sessiz* olduğudur. Sistem çalışma sırasında herhangi bir hata (Exception/Error) fırlatmaz, kod çökmez ve süreç başarıyla tamamlanmış gibi görünür. Ancak arka planda model; kaybolan kelimeleri, aşırı şişmiş bağlam pencerelerini (context window) ve kaymış varlık (entity) konumlarını doğru kabul ederek öğrenmeye ve çıkarım yapmaya devam eder. Bu durum, eğitim maliyetlerini artırırken modelin başarımını (accuracy) dramatik şekilde düşürür.

2. Deney Düzeni ve Kapsam

Tokenizer'ların İngilizce ve Türkçe üzerindeki davranışlarını bilimsel ve yansız bir şekilde kıyaslayabilmek için deney ortamı standartlaştırılmıştır. Hugging Face kütüphaneleri (Transformers, Datasets, Tokenizers) kullanılarak en yaygın üç alt-kelime (subword) algoritması seçilmiş ve her biri hem İngilizce hem de Türkçe derlemler üzerinde sıfırdan eğitilmiştir.

Eğitimlerde kelime dağarcığı (vocabulary) boyutu her algoritma ve dil için **8.000 token** olarak sabit tutulmuştur. Bu sayede, performans farklılıklarının dağarcık boyutundan değil, doğrudan algoritma ve dilin doğasından kaynaklandığından emin olunmuştur.

Bileşen	Seçim ve Detaylar
Algoritmalar	WordPiece (BERT), BPE (GPT-2, bayt düzeyinde), Unigram (XLNet / SentencePiece)
İngilizce Derlem	UD English-EWT: 12.544 eğitim, 2.077 test cümlesi
Türkçe Derlem	UD Turkish-BOUN: 7.803 eğitim, 979 test cümlesi
Tokenizer Sayısı	3 algoritma × 2 dil = Toplam 6 izole tokenizer modeli
Kelime Dağarcığı	Tüm tokenizer'lar için tam 8.000 token (özel token'lar dahil)
Ana Ölçüt	**Doğurganlık (Fertility):** Kelime başına düşen ortalama token sayısı. Düşük değer, tokenizer'ın o dili daha verimli sıkıştırdığını gösterir.

3. Kontrol Testi: Temel Değerler (Baseline)

Deneylerin tutarlılığını ölçmek adına ilk olarak her tokenizer kendi eğitildiği dildeki test metinleri üzerinde sınanmıştır (İngilizce tokenizer İngilizce metinde, Türkçe tokenizer Türkçe metinde). Buradan elde edilecek "doğurganlık" skorları, sonrasında yapılacak çapraz dil testleri için bir referans (baseline) noktası oluşturacaktır.

Doğurganlık, bir dildeki ortalama bir kelimenin, tokenizer tarafından kaç adet alt-kelime (token) parçasına bölündüğünü ifade eder. İdeal bir senaryoda bu sayının 1.0'e yakın olması istenir (her kelimenin tek bir token ile temsil edilmesi), ancak alt-kelime algoritmalarının doğası gereği bu değer genellikle 1.5 ile 2.5 arasında değişir.

Algoritma	EN Tokenizer → EN Metin	TR Tokenizer → TR Metin
WordPiece	1.53 token / kelime	1.85 token / kelime
BPE (Byte-Level)	1.61 token / kelime	1.99 token / kelime
Unigram	1.63 token / kelime	1.97 token / kelime

Tablodan görüleceği üzere Türkçe değerlerin İngilizceye kıyasla bir miktar daha yüksek olması beklenen bir durumdur. Türkçe, eklemeli (agglutinative) bir dil olduğu için kelimeler sondan ek alarak (Örn: *git-ti-k-çe*) İngilizcedeki bir cümleye denk düşecek kadar uzayabilir. Bu nedenle Türkçe bir kelime, İngilizce bir kelimeden daha fazla bilgi yoğunluğu taşır ve doğal olarak daha fazla alt parçaya bölünür.

4. Test 1: Dil Uyumsuzluğu ve Token Şişmesi

Tehdit Tanımı: Türkçede faaliyet gösteren bir projede, halihazırda eğitilmiş, İngilizce merkezli güçlü bir Açık Kaynak modelin (Örn: Llama, GPT-2 veya BERT) varsayılan tokenizer'ı ile doğrudan kullanılması senaryosudur.

Yöntem: Yalnızca İngilizce derlem üzerinde eğitilmiş olan tokenizer'lara, tamamen Türkçe bir metin verilmiş ve doğurganlık (parçalanma) oranlarındaki artış izlenmiştir. Beklenti, İngilizce tokenizer'ın Türkçeye özgü harfleri (ş, ğ, ı, ç, ö, ü) ve yaygın Türkçe ekleri (lerimiz, mış, dan) tanımadığı için kelimeleri gereğinden fazla küçük parçalara, hatta tek tek harflere bölmesidir.

Algoritma	Türkçe Tok. Skoru	İngilizce Tok. Skoru	Artış (Maliyet Çarpanı)
WordPiece	1.85	2.80	1.51x
BPE (Byte-Level)	1.99	5.06	**2.54x**
Unigram	1.97	4.88	**2.48x**

Sonuçlar çarpıcıdır. GPT-2 tarzı bir bayt düzeyinde BPE algoritması veya Unigram, aynı Türkçe metni temsil etmek için **yaklaşık 2.5 kat daha fazla** token üretmektedir. İngilizce derlemde "ı", "ş" gibi harfler nadir bulunduğu için tokenizer bu harfleri içeren alt kelimeleri birleştirmeyi öğrenmemiştir. Hatta BPE mimarisinde her Türkçe (non-ASCII) karakter 2 bayt yer kapladığı için, tokenizer bunları tek bir harf olarak bile görmekte zorlanır ve anlamsız bayt dizilerine böler.

5. Test 1'in Etkileri: Bağlam Daralması ve Maliyet

Token sayısındaki bu dramatik artış, (2.5x çarpanı) sadece akademik bir metrik değildir. Doğrudan donanım maliyeti, API ücretlendirmesi ve model hafızası ile ilgilidir. Klasik bir Transformer modelinin genellikle 512, 2048 veya 8192 token ile sınırlı bir bağlam penceresi (context window) bulunur. Tokenizer, kelimeleri ne kadar verimsiz parçalarsa, modele o kadar az bilgi sığdırılabilir.

Bağlam Daralması (Context Window Shrinking) Örneği: 512 token limitli bir girdi düşünelim.
- *Türkçe için özel eğitilmiş BPE:* ~257 kelime sığdırabilir.
- *İngilizce odaklı BPE:* Aynı modele yalnızca ~101 kelime sığdırabilir.
Modelin görme kapasitesi %60 oranında daralmıştır.

Maliyetin ötesinde, Türkçe dil bilgisi ve anlamsal bütünlük de zarar görür. Örneğin *"Çekoslovakyalılaştıramadıklarımızdanmışsınız"* kelimesi Türkçe bir BPE tokenizer ile 13 mantıklı heceye (ek kök yapısına) bölünürken, İngilizce BPE bunu **44 anlamsız parçaya** ayırır. Model, dikkat (attention) mekanizmasını bu 44 dağınık parça üzerinde kurmaya çalışırken cümlenin ana fikrini kaybeder.



6. Test 2: Dağarcık Eksikliği ve Bilgi Kaybı

Tehdit Tanımı: Modelin, kendisine verilen girdideki bazı kelimeleri kısmen veya tamamen "Bilinmeyen" olarak sınıflandırması ve o kelimenin taşıdığı bilgiyi tamamen kaybetmesi.

WordPiece ve BPE algoritmaları, bilinmeyen karakterlere tamamen farklı yaklaşımlar sergiler. Bayt düzeyinde BPE (GPT-2 stili), teorik olarak evrendeki tüm karakterleri 256 adet temel baytın birleşimi olarak ifade edebildiği için hiçbir zaman "Bilinmeyen" `[UNK]` token'ı üretmez. Ancak WordPiece (BERT stili), eğer bir kelimenin içindeki tek bir harfi bile kelime dağarcığında (vocab) bulamazsa, o **kelimenin tamamını** çöpe atarak `[UNK]` işareti basar.

Algoritma	Üretilen [UNK]	Toplam Token	[UNK] Oranı
WordPiece	2.127	27.932	**%7.61**
BPE (Byte-Level)	0	50.577	%0.00
Unigram	0	48.731	%0.00

Tablodaki %7.61'lik oran, üretilen tüm token'lar içindeki bilinmeyen oranını ifade etmektedir. Ancak bu durum, cümledeki "farklı kelimelerin" kaybı açısından incelendiğinde tablonun çok daha vahim olduğu ortaya çıkar.

7. Test 2'nin Etkileri: [UNK] Yıkımı

İngilizce derlemle eğitilmiş WordPiece tokenizer'ı Türkçe test metnine uygulandığında, test kümesindeki 6.686 farklı kelimeden tam **1.231 tanesi (%18.4)** tamamen `[UNK]` olmuş ve yok olmuştur. Peki bu kaybolan kelimelerin ortak noktası nedir?

I/ı Harfinin Trajedisi: Yapılan analizde, kaybolan 1.231 kelimenin **%100.0'ının** "ı" veya "I" harfi içerdiği tespit edilmiştir. İngilizcede "ı" harfi bulunmaz. Bu harf dağarcığa hiç giremediği için, içinde "ı" geçen her kelime WordPiece tarafından okunamaz ilan edilmiştir.

Kelime	WordPiece (İngilizce Eğitilmiş)	Unigram (Türkçe Eğitilmiş)
ışık	`['[UNK]']`	`['▁', 'ışı', 'k']`
kılıç	`['[UNK]']`	`['▁', 'k', 'ı', 'l', 'ı', 'ç']`
ağaç	`['ag', '##ac']`	`['▁a', 'ğ', 'a', 'ç']`

Yukarıdaki tabloda görüldüğü üzere model, kendisine "ışık" veya "kılıç" kelimeleri verildiğinde ikisini de `[UNK]` olarak görecek ve aralarındaki farkı ayırt edemeyecektir. BPE veya Unigram algoritmaları bu kelimeleri yaşatmayı başarsa da, bu kez Test 1'de gördüğümüz maliyet şişmesi sorunuyla karşılaşmaktadırlar.

8. Test 3: Standart Normalizasyon Sorunu

Tehdit Tanımı: Modelin öncesinde çalışan normalizasyon (temizlik) adımının, o dilin kurallarını bilmemesi nedeniyle kelimeleri ve anlamları birbirine karıştırmasıdır. En belirgin örnek, `bert-base-uncased` modelinin kullandığı İngilizce odaklı `BertNormalizer` mimarisidir.

Unicode'un standart İngilizce küçük harf çevrimi kuralı Türkçeyi bilmez. İngilizcede büyük **"I"** harfi küçültüldüğünde **"i"** olur (doğrusu Türkçede "ı" olmalıdır). Ayrıca İngilizcede büyük **"İ"** (noktalı) bulunmadığından, bunu dönüştürürken "i" harfi ve ardına "birleşik nokta" ekleyerek iki ayrı karakter oluşturur. Bunun üzerine bir de aksan silme (strip accents) özelliği devredeyse, ç→c, ö→o, ş→s, ğ→g dönüşümleri yaşanır.

Kelime 1	Kelime 2	BertNormalizer Çıktısı	Doğru TR Normalizer Çıktısı	Sonuç Farkı
SINIR	sinir	sinir / sinir	sınır / sinir	**Hatalı (Birleştiler)**
ILIK	ilik	ilik / ilik	ılık / ilik	**Hatalı (Birleştiler)**
göl	gol	gol / gol	göl / gol	**Hatalı (Birleştiler)**
çam	cam	cam / cam	çam / cam	**Hatalı (Birleştiler)**
IŞIK	ışık	isik / ısık	ışık / ışık	**Hatalı (Bölündüler)**
SINIR	sınır	sinir / sınır	sınır / sınır	**Hatalı (Bölündüler)**

9. Test 3'ün Etkileri: Anlam Çakışması Analizi

Tablodaki örnekler tüm derlem (yaklaşık 32.631 farklı kelime) ölçeğinde uygulandığında, dil bilmeyen bir normalizasyon sisteminin iki büyük yapısal yıkıma yol açtığı görülmüştür.

1. Farklı Kelimelerin Birleşmesi (Çakışma):
İngilizce normalizasyon, aksanları sildiği için **214 farklı Türkçe kelimeyi** aynı token formuna indirgemiştir. Model; "açısından" ile "acısından", "başına" ile "basına", "sınır" ile "sinir" arasındaki farkı asla göremez. Cümle içinde bu kelimeler tamamen aynı sayısal ID ile temsil edilir.

2. Aynı Kelimenin İkiye Bölünmesi (Dağılma):
Kelimelerin yaklaşık **%30.1'inde** (9.836 kelime), sırf BÜYÜK veya küçük yazıldıkları için farklı kelimeler zannedilmiştir. Örneğin bir haber başlığında geçen `ABARTMIŞ` (abartmis olarak normalize edilir) ile metin içinde geçen `abartmış` (abartmıs olarak kalır) kelimeleri ayrı token ID'leri alarak modelin aynı kavram üzerinde öğrenmesini engeller.

Türkçe kurallı bir normalizer (I→ı, İ→i ve aksanları koruma) uygulandığında bu iki sorunun da hata oranı **sıfıra** inmektedir. Çözüm, modeli değiştirmekten değil, modelin giriş kapısındaki temizlik birimini o dile özgü inşa etmekten geçmektedir.

10. Test 4: Filtre Atlatma (Güvenlik)

Tehdit Tanımı: Çevrimiçi bir platformun, yasaklı (küfür, hakaret vb.) bir kelimeyi token ID'lerine bakarak engelleyen basit bir filtresi olsun. Kötü niyetli bir kullanıcı, kelimeyi *insan gözüne tamamen aynı görünecek* şekilde gizleyerek filtreyi atlatmayı dener.

Bu senaryoda, hedef kelime olan "dolandırıcı" için üç farklı gizleme (obfuscation) varyantı oluşturulmuş ve farklı normalizasyon savunmaları test edilmiştir:

Saldırı Türü	Nasıl Yapılır?	Görünüş
Kiril Harfleri	Latin 'a' ve 'o' harfleri yerine, Rus Kiril alfabesindeki birebir aynı görünen 'a' (U+0430) ve 'o' (U+043E) kullanılır.	`dolandırıcı`
Sıfır Genişlikli Boşluk	Kelimedeki her harfin arasına görünmez bir karakter olan (U+200B) yerleştirilir.	`dolandırıcı`
Tam Genişlikli Harfler	Genellikle Asya fontlarında kullanılan daha geniş harf setleri (U+FF41–U+FF5A) kullanılır.	`ｄｏｌａｎｄｉｒｉｃｉ`

11. Test 4'ün Sonuçları: Savunma Performansı

Aşağıdaki tablo, oluşturulan üç saldırı türünün farklı tokenizer ve normalizasyon mimarilerine karşı nasıl sonuçlandığını göstermektedir (Başarı: Saldırının atlatılması, Yakalandı: Savunmanın çalışması).

Savunma Türü	Kiril 'a' ve 'o'	Sıfır Genişlik	Tam Genişlik
Normalizer Yok (GPT-2)	**Atlatıldı**	**Atlatıldı**	**Atlatıldı**
BertNormalizer	**Atlatıldı**	**Yakalandı**	**Atlatıldı**
Standart NFKC	**Atlatıldı**	**Atlatıldı**	**Yakalandı**
Türkçe Normalizer (Manuel)	**Atlatıldı**	**Yakalandı**	**Yakalandı**
Türkçe + Kiril Eşleme Tablosu	**Yakalandı**	**Yakalandı**	**Yakalandı**

Değerlendirme: Hiçbir standart Unicode normalizasyonu (NFKC dahil) Kiril harflerini yakalayamaz, çünkü teknik olarak bunlar Latin harflerinden tamamen farklı dizinlerdeki "farklı" harflerdir. Tam teşekküllü bir güvenlik için tokenizer hattına "Benzer Harf" (Confusables) eşleme tablosu eklenmesi şarttır.

12. Hızlı Tokenizer'ların Süper Güçleri

Hugging Face ekosisteminde Python ile yazılmış "Yavaş" (Slow) ve Rust ile yazılmış "Hızlı" (Fast) tokenizer'lar bulunur. Hızlı tokenizer'ların tek avantajı çoklu iş parçacığıyla metinleri saniyeler içinde işleyebilmesi değildir; asıl güçleri, metnin yapısal bütünlüğünü arka planda takip edebilmeleridir.

Bir Hızlı Tokenizer, parçaladığı (tokenize ettiği) bir metnin ID'lerini geri döndürürken, yanında şu yetenekleri sağlayan özel bir `BatchEncoding` nesnesi sunar:

- **Offset Eşlemesi (Offset Mapping):** Nihai üretilen her bir token'ın, orijinal metindeki tam karakter aralığını (span) daima hatırlar. `return_offsets_mapping=True` ile her tokenin (başlangıç, bitiş) indeksi öğrenilebilir.
- **Kelime İzleme:** `word_ids()` fonksiyonu, üretilen hece parçalarının asıl metindeki hangi bütün kelimeden geldiğini söyler.
- **Uzun Bağlam Yönetimi (Stride):** Metin modelin sınırını aşıyorsa `return_overflowing_tokens=True` ile metni parçalara ayırır, ancak parçalar arasında istenen miktarda örtüşme (stride) bırakarak kesişim noktalarındaki verinin kaybolmasını engeller.

Bu özellikler özellikle Soru-Cevap (Question Answering) ve Varlık İsmi Tanıma (NER) gibi görevlerde, modelin bulduğu matematiksel konumu, okuyucunun gördüğü gerçek metin konumuna geri çevirmek için hayati önem taşır.

13. Test 5: Dış Normalizasyon ve Offset Kayması

Tehdit Tanımı: Makine öğrenmesi projelerinde geliştiricilerin sıkça yaptığı bir hata, metni tokenizer'a vermeden hemen önce Python'un standart `.lower()` metoduyla küçük harfe çevirmektir. Dışarıda yapılan bu işlemin Türkçe üzerindeki etkisi, offset (konum) eşlemesini tamamen bozmasıdır.

Python'un standart string metodu, Türkçedeki büyük **"İ"** harfini küçültürken, "i" karakteri ve ardından birleşik bir nokta karakteri olmak üzere **iki karakter** üretir. Yani metin içinde geçen her "İ" harfi, cümlenin toplam uzunluğunu (karakter sayısını) bir adım uzatır.

Zincirleme Hata: Metin tokenizer'a *dışarıda uzatılmış* haliyle girer. Tokenizer, Hızlı Süper Gücü sayesinde offset'leri hesaplar. Ancak geliştirici, modelin döndürdüğü offset rakamlarını alıp *orijinal (uzatılmamış) metin* üzerinde dilimleme (slicing) yaparsa, "İ" harfinden sonraki tüm kelimeler için konumlar kaymış olacaktır.

Eğer normalizasyon işlemi tokenizer'ın *içindeki* Normalizer bloğuna bırakılırsa, tokenizer bu karakter değişimlerini dahili olarak takip eder ve offset kaymasına asla izin vermez. Ancak dışarıdan yapılan bir müdahale bu zırhı deler.

14. Test 5'in Etkileri: NER Pipeline Çöküşü

Offset kayması yaşandığında, Varlık Çıkarımı (NER) yapan bir modelin bulduğu sonuçların orjinal metinden nasıl yanlış koparıldığını aşağıdaki tabloda görebilirsiniz. Örnek cümle: *"İSTANBUL'da İzmirli öğrenci ödül kazandı."* (Orijinal uzunluk: 45, .lower() sonrası uzunluk: 47)

Modelin Bulduğu Token	İÇERİDE Normalizasyon (Doğru Kesim)	DIŞARIDA .lower() (Hatalı Kesim)
istanbul	`İSTANBUL`	`İSTANBUL'`
'	'	`d`
da	`da`	`a`
izmir	`İzmir`	`zmirli`
##li	`li`	`b`
ogrenci	`öğrenci`	`renci ö`
odul	`ödül`	`ül k`
kazandı	`kazandı`	`zandı.`

Derlem ölçeğinde test edildiğinde, "İ" harfi içeren tüm cümlelerde dışarıda `.lower()` kullanılması, tokenların **%80.6'sının** metinde yanlış yeri (örn: "İzmir" yerine "zmirli" bölgesini) işaret etmesine neden olmuştur. İçeride normalizasyon kullanıldığında hata oranı %0'dır.

15. Bonus Test 6: Eğitim Determinizmi

Hugging Face eğitim materyallerinde yer alan bir diğer iddia: *"Tokenizer eğitimi deterministiktir; aynı algoritmayla aynı derlemde eğitim yaptığınızda her zaman aynı sonuçları alırsınız."* cümlesidir. Bu iddia, bilimsel tekrar edilebilirlik (reproducibility) açısından çok kritiktir.

Aynı İngilizce derlem ve aynı ayarlar kullanılarak BPE ve WordPiece algoritmaları 4'er kez baştan sona eğitilmiş ve elde edilen kelime dağarcıkları (8.000 token) birbiriyle kıyaslanmıştır.

Algoritma	4 Eğitimin Hepsi Aynı Mı?	1. Eğitimle Farklı Token Sayısı (2., 3., 4. deneme)
BPE (Byte-Level)	**Evet (True)**	[0, 0, 0]
WordPiece	**Hayır (False)**	[16, 16, 18]

Sonuçlar BPE için iddiayı doğrulamakta, ancak WordPiece için yalanlamaktadır. WordPiece eğiticisi, çok sayıda rastgelelik içermese de, eşit frekanslı parçalar arasında seçim yaparken iç veri yapılarındaki sıralamaya dayanır. Etkisi küçük (8.000 tokende 15-20 fark) olsa da **birebir aynı modelin** elde edilmesini engeller. Bu nedenle eğitilen `tokenizer.json` dosyası her zaman kaydedilmeli ve projeye dahil edilmelidir.

16. Algoritma İncelemesi 1: Byte-Pair Encoding (BPE)

Byte-Pair Encoding (BPE), metinleri sıkıştırmak için geliştirilmiş bir veri sıkıştırma algoritması iken OpenAI tarafından GPT serisi (GPT-2, RoBERTa, BART vb.) modellerin tokenizasyon süreçlerine uyarlanmıştır.

Eğitim Süreci: BPE, eğitime metinlerdeki kelimeleri en küçük yapı taşına, yani karakterlerine ayırarak başlar. (Bayt düzeyinde BPE ise Unicode karakterleri yerine 256 temel baytı kullanır). Ardından şu döngüyü işletir:

1. Mevcut karakter/hece listesinde yan yana *en çok tekrar eden* ikiliyi (pair) bul.
2. Bu ikiliyi tek bir token olarak birleştir (Örn: "u" ve "g" yan yana çok geçiyorsa onları "ug" yap).
3. İstenen toplam kelime dağarcığı (vocabulary) boyutuna ulaşana kadar bunu tekrar et.

Kodlama (Encoding): Yeni bir kelime geldiğinde, kelime önce harflerine ayrılır. Ardından, eğitim sırasında öğrenilen birleştirme kuralları (merge rules) *aynı sırayla* bu harflere uygulanarak kelime parçalanır.

Bayt tabanlı BPE'nin en büyük avantajı `[UNK]` (bilinmeyen) token'ı oluşturma ihtimalinin sıfır olmasıdır. Ancak dezavantajı, Türkçede Test 1'de gördüğümüz gibi, tanımadığı harf ve heceleri birleştirmeyi bilmediği için verimsiz bir şekilde tek tek baytlara bölebilmesidir.

17. Algoritma İncelemesi 2: WordPiece

WordPiece, Google tarafından özellikle BERT modeli (ve DistilBERT, MobileBERT gibi varyantları) için geliştirilmiştir. Temel yapısı itibarıyla BPE'ye çok benzer ancak yeni bir token oluştururken "yan yana en çok geçme" metriğini değil, istatistiksel bir skorlama formülünü kullanır.

BPE sadece frekansa bakarken, WordPiece şu formüle göre birleştirme kararı alır:

> Skor = (İkilinin Frekansı) / (1. Parçanın Frekansı × 2. Parçanın Frekansı)

Neden böyle bir formül var? Bu formül, kelime dağarcığında tek başlarına çok sık geçen ama yan yana geldiklerinde de görece sık görülen (örneğin "t" ve "e" harfleri) parçaları birleştirmek yerine, tek başlarına nadir bulunan ama *yan yana geldiklerinde anlam ifade eden* (örneğin "hugg" ve "##ing") parçaları birleştirmeye öncelik verir. WordPiece, kelime içindeki parçaların başına ## işaretini koyarak kelime başı ile kelime içini ayırt eder.

Kodlama aşamasında (Encoding) BPE gibi birleştirme kurallarını sırayla işletmez. Kelime dağarcığında baştan başlayarak uyan *en uzun* parçayı bulur (açgözlü / greedy yaklaşım) ve kelimeyi oradan keser. Herhangi bir parça dağarcıkta bulunamazsa kelimenin tamamını `[UNK]` yapar.

18. Algoritma İncelemesi 3: Unigram

Unigram algoritması, T5, ALBERT ve XLNet gibi modellerin tokenizasyon motorudur. Genellikle boşlukların kelime ayracı olarak kullanılmadığı dilleri destekleyen *SentencePiece* sistemiyle birlikte kullanılır.

Unigram, BPE ve WordPiece'in **tam tersi** yönde çalışır. Onlar küçük bir harf havuzuyla başlayıp parçaları büyüterek hedefe ulaşırken, Unigram devasa bir olasılık havuzuyla (tüm olası kelime parçalarıyla) işe başlar ve budama (pruning) yöntemiyle küçülür.

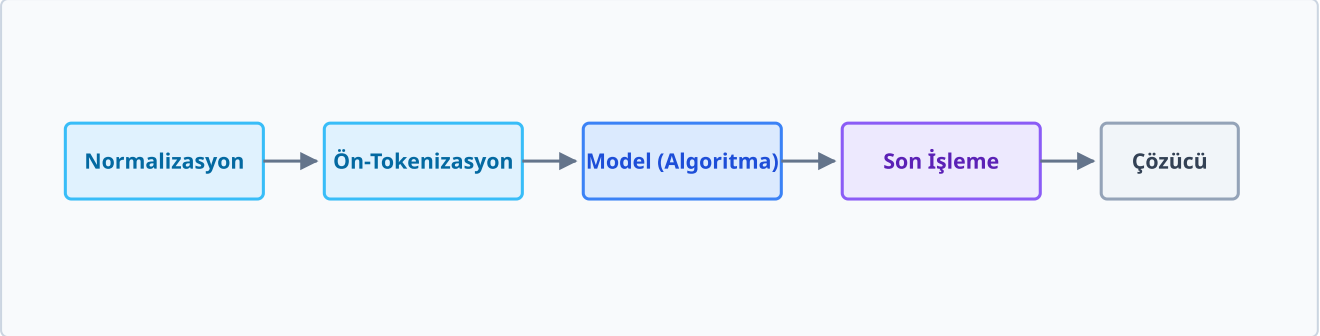
Eğitimin her adımında Unigram, tüm derlem üzerindeki "Kayıp" (Loss / Negative Log Likelihood) değerini hesaplar. Ardından her bir token'ı havuzdan çıkardığında kaybın ne kadar kötüleştiğine (arttığına) bakar. Çıkarıldığında kaybı en az etkileyen tokenlar "gereksiz" kabul edilir ve %10-20'lik gruplar halinde dağarcıktan silinir.

Algoritma Türü	Eğitim Yönü	Uygulama Yöntemi
BPE / WordPiece	Aşağıdan Yukarı (Büyüme)	Kuralları Sırayla Uygulama / Açgözlü
Unigram	Yukarıdan Aşağı (Budama)	Viterbi Olasılık Algoritması

Unigram kodlama yaparken (örneğin "pug" kelimesini parçalarken) tek bir doğru yol izlemez. ["p", "u", "g"] ve ["pu", "g"] gibi tüm olası parçalanma varyasyonlarının olasılıklarını hesaplar. Hangi kombinasyon istatistiksel olarak en muhtemel ise onu seçer. Bu, esnek ve oldukça başarılı bir yaklaşımdır.

19. Tokenizer'ı Sıfırdan ve Blok Blok İnşa Etmek

Hugging Face `Tokenizers` kütüphanesi, yepyeni bir algoritma yaratabilmek için birbirine takılıp çıkarılabilen yapı taşları (modüller) sunar. Modern bir NLP boru hattında ham metnin bir modelin anlayabileceği ID'lere dönüşmesi şu bloklardan geçer:



- **1. Normalizers:** Metnin temizliğidir. Unicode eşitleme, küçük harfe alma veya aksanları kaldırma burada yapılır.
- **2. Pre-tokenizers:** Cümleleri kabaca kelimelere ayırır (boşluklar veya noktalama işaretleri ile).
- **3. Models:** Esas beyindir. BPE, WordPiece veya Unigram bu bloktadır ve alt-kelimelere ayırır.
- **4. Post-processors:** `[CLS]` ve `[SEP]` gibi özel başlangıç ve bitiş sembollerini ekler.
- **5. Decoders:** ID'leri okuyabileceğimiz metin formatına geri çeviren (örneğin ## işaretlerini temizleyen) bölümdür.

20. Sonuç ve Güvenli Türkçe Mimarisi İçin Öneriler

Gerçekleştirilen Red-Teaming testleri, bir modelin başka bir dildeki (İngilizce) ön yargılarıyla Türkçeye yaklaştığında nasıl sessizce maliyet ürettiğini ve veriyi yok ettiğini açıkça kanıtlamıştır. Gelecekteki Türkçe NLP projelerinde bu zafiyetleri önlemek için aşağıdaki savunma mimarisi benimsenmelidir:

- **Yeniden Eğitim (Re-training):** İngilizce odaklı (Örn: Llama, GPT) hazır bir model doğrudan kullanılmamalı, Türkçe metindeki doğurganlığı test edilmelidir. Şişme oranı 2x'in üzerindeyse tokenizer, Türkçe derlemle yeniden eğitilmelidir.
- **Dahili Normalizasyon:** Veri ön işleme adımlarında (dışarıda) `.lower()` kullanılmamalıdır. Bu işlem sadece tokenizer'ın kendi bloğu içinde yapılmalıdır ki Offset (konum) eşlemeleri kaymasın.
- **Aksanların Korunması:** `strip_accents` özelliği Türkçede kesinlikle kapatılmalıdır. (I/İ) dönüşümü için özel bir Türkçe normalizasyon bloğu yazılmalıdır.
- **Sıfır Güven Filtrelemesi:** Siber güvenlik tarafında küfür/yasaklı kelime filtreleri kuruluyorsa, token bazlı arama yapılmadan önce Kiril eşleme tablosu ve görünmez karakter (Zero-width) temizleyicisi hatta eklenmelidir.

Son Söz: Tokenizer, yapay zeka modelinin "gözleri"dir. Gözü bozuk olan bir model ne kadar büyük ve güçlü olursa olsun, okuduğu metni yanlış anlayacak ve maliyetli hatalar üretecektir. Tokenizer seçimi, model seçimi kadar kritiktir.